that depends intimately on the bits of i, looping over many values of i. Then, if there are economies in repeating the task for values differing by only one bit, it makes sense to do things in Gray code order rather than consecutive order. We saw an example of this in §7.8, for the generation of quasi-random sequences.

CITED REFERENCES AND FURTHER READING:

Horowitz, P., and Hill, W. 1989, *The Art of Electronics*, 2nd ed. (New York: Cambridge University Press), §8.02.

Knuth, D.E. 2005, *Generating All Tuples and Permutations*, fascicle 2 of vol. 4 of *The Art of Computer Programming* (Upper Saddle River, NJ: Addison-Wesley), §7.2.1.1.

22.4 Cyclic Redundancy and Other Checksums

There are networks all around you: not just "the" Internet with its IP and TCP protocols, but also *embedded* networks that move bits around within a device or among closely coupled devices. Examples include the *SMBus* network that communicates power management information between smart batteries and the devices that they power, or the *Bluetooth* network that connects cell phones to nearby accessories. We wouldn't be overly surprised to find a network inside of our wristwatch or electric toothbrush!

Different networks have different protocols, but standard engineering practice is to package the raw information into packets with fixed or variable numbers of bits. Packet lengths are typically in the range from a few tens to a few thousand bits. Smaller would imply proportionally too much overhead per packet, while longer would make excessive demands on buffer sizes, collision avoidance, etc.

When a packet is sent from point A to point B, one wants to know that it has arrived without error. The simplest form of insurance is to add a "parity bit," chosen so as to make the total number of one-bits (versus zero-bits) either always even ("even parity") or always odd ("odd parity"). Any *single-bit* error in a packet will thereby be detected. When errors are sufficiently rare, or their consequence sufficiently minor, use of parity provides enough error detection. For example, the ASCII character set was originally designed for 7-bit characters, with an 8th parity bit.

Since the parity bit has two possible values (0 and 1), it has, on average, only a 50% chance of detecting an erroneous packet with *multiple* wrong bits. That is not nearly good enough for most applications. Most communications protocols [1] use a multibit generalization of the parity bit called a "cyclic redundancy check" or CRC. Often, the CRC is 16 bits (two bytes) long. Then the chance of a random set of errors going undetected is 1 in $2^{16} = 65536$.

Now enters mathematics. It is easy to find M-bit CRCs that have the property of detecting *all* errors that occur in M or fewer *consecutive* bits, for any length of message. (We prove this below.) Since noise in communication channels tends to be "bursty," with short sequences of adjacent bits getting corrupted, this consecutive-bit property is highly desirable. Furthermore, for packets with a fixed (or bounded) payload size of N bits, one can find CRCs that find all occurrences of D or fewer errors *anywhere* in the payload. Obviously, the game is to find the CRC that maximizes D. The value $D + 1$ is the *Hamming distance* of the CRC for that value of N using that checksum (cf. §16.2).

Useful 16-bit CRC Polynomials (after [3])			
j	Name	Polynomial	Best N (bits)
0		`0x755B` $= (x^3+x^2+1)^*(x^6+x^5+x^2+x+1)^*(x^7+x^3+1)^*$	242–2048+
1		`0xA7D3` $= (x^3+x^2+1)^*(x^6+x^5+x^2+x+1)^*(x^7+x^6+x^5+x^4+1)^*$	256–2048+
2	ANSI-16	`0x8005` $= (x+1)(x^{15}+x+1)^*$	242–2048+
3	CCITT-16	`0x1021` $= (x+1)(x^{15}+x^{14}+x^{13}+x^{12}+x^4+x^3+x^2+x+1)^*$	242–2048+
4		`0x5935` $= (x^{16}+x^{14}+x^{12}+x^{11}+x^8+x^5+x^4+x^2+1)$	136–241
5		`0x90D9` $= (x+1)(x^{15}+x^{11}+x^{10}+x^9+x^8+x^7+x^5+x^4+x^2+x+1)$	20–135
6	IEC-16	`0x5B93` $= (x+1)(x+1)(x^7+x^6+x^3+x+1)^*(x^7+x^6+x^5+x^4+x^3+x^2+1)^*$	20–112
7		`0x2D17` $= (x^2+x+1)^*(x^{14}+x^{13}+x^9+x^7+x^5+x^4+1)$	16–19
* denotes primitive factor			

The design of CRCs lies in the province of communications software experts and chip-level hardware designers — people with bits under their fingernails. A passing familiarity with some of the concepts involved can be useful, however, both because the mathematics involved has connections to other applications (for example, random number generation, cf. §7.1 and §7.5), and because you might actually want to add a couple of bytes of checksum to your own data records in some applications where you are handling, or moving, large amounts of data.

Sometimes CRCs can be used to compress data as they are recorded. If identical data records occur frequently, one can keep sorted in memory the CRCs of previously encountered records. A new record is archived in full if its CRC is different, otherwise only a pointer to a previous record need be archived. In this application one might use 8 bytes of CRC, to make the odds of mistakenly discarding a different data record tolerably small; or, if previous records can be randomly accessed, a full comparison can be made to decide whether records with identical CRCs are in fact identical.

Now let us briefly discuss the theory of CRCs. After that, we will give an implementation that generates 16-bit CRCs that are known to be particularly good, or else are enshrined as standard (and, it turns out, this is not the same thing!).

The mathematics underlying CRCs is "polynomials over the integers modulo 2." Any binary message can be thought of as a polynomial with coefficients 0 and 1. For example, the message "1100001101" is the polynomial $x^9 + x^8 + x^3 + x^2 + 1$. Since 0 and 1 are the only integers modulo 2, a power of x in the polynomial is either present (1) or absent (0).

An M-bit-long CRC is based on a polynomial of degree M, called the generator polynomial. Given the generator polynomial G (which can be written either in polynomial form or as a bit-string, e.g., 10001000000100001 for $x^{16} + x^{12} + x^5 + 1$), here is how you compute the CRC for a sequence of bits S: First, multiply S by x^M, that is, append M zero bits to it. Second, divide, by long division, G into Sx^M. Keep in mind that the subtractions in the long division are done modulo 2, so that there are never any "borrows": Modulo 2 subtraction is the same as logical exclusive-or (XOR). Third, ignore the quotient you get. Fourth, when you eventually get to a remainder, it is the CRC; call it C. C will be a polynomial of degree $M - 1$ or less, otherwise you would not have finished the long division. Therefore, in bit-string form, it has M bits, which may include leading zeros. (C might even be all zeros; see below.)

If you work through the above steps in an example, you will see that most of what you write down in the long-division tableau is superfluous. You are actually just left-shifting sequential bits of S, from the right, into an M-bit register. Every time a 1 bit gets shifted off the left end of this register, you zap the register by an XOR with the M low-order bits of G (that is, all the bits of G except its leading 1). When a 0 bit is shifted off the left end you don't zap the register. When the last bit that was originally part of S gets shifted off the left end of the register, what remains is the CRC.

You can immediately recognize how efficiently this procedure can be implemented in hardware. It requires only a shift register with a few hard-wired XOR taps into it. That is how CRCs are computed in communications devices, taking a tiny part of a single chip. In software, the implementation is not so elegant, since bit-shifting is not generally very efficient. One therefore typically finds (as in our implementation below) table-driven routines that pre-calculate the result of a bunch of shifts and XORs, say for each of 256 possible 8-bit inputs [2].

Every generator polynomial of degree M with a nonzero x^0 term yields a CRC that detects *all* possible combinations of errors in any *frame* of M consecutive bits. (A special case of this is that it detects any single-bit error in a message of arbitrary length N.) To see how this works, suppose two messages, S and T, differ only within a frame of M bits. Then their CRCs differ by an amount that is the remainder when G is divided into $(S - T)x^M \equiv R$. Now R has the form of leading zeros (which can be ignored), followed by some 1's in an M-bit frame, followed by trailing zeros (which are just multiplicative factors of x): $R = x^n F$, where F is a polynomial of degree at most $M - 1$ and $n > 0$. But since G has a nonzero x^0 term, it is not divisible by x. So G cannot divide R. Therefore S and T must have different CRCs.

What about two-bit errors, not necessarily in a frame of size M? That leads us to *primitive polynomials*: A polynomial over the integers modulo 2 may be irreducible, meaning that it can't be factored. A subset of the irreducible polynomials is the primitive polynomials. These generate maximum length sequences when used in shift registers, as described in §7.5. The polynomial $x^2 + 1$ is not irreducible: $x^2 + 1 = (x + 1)(x + 1)$, so it is also not primitive. The polynomial $x^4 + x^3 + x^2 + x + 1$ is irreducible, but it turns out not to be primitive. The polynomial $x^4 + x + 1$ is both irreducible and primitive.

Primitive polynomials are here interesting because they have a very high order. Don't confuse *order* with *degree*: The order e of a polynomial is the smallest integer e such that the polynomial divides (in the present mod 2 case) $x^e + 1$. Primitive

polynomials, it turns out, have the largest possible order e for their degree n, given by

$$e = 2^n - 1 \tag{22.4.1}$$

(In fact, this is why their shift registers have maximum length.) If two messages differ on exactly two bits, spaced k bits apart, then their difference is $x^k + 1$ times some trailing powers of x. If the generator G contains a primitive factor of order e, then G can't possibly divide this difference, as long as $k < e$.

Thus, a primitive factor of degree n guarantees two-bit error detection for spacings up to $2^n - 1$. For this reason, generators are often chosen to be primitive polynomials of degree M. Alternatively, the generator may be chosen be a primitive polynomial times $(1 + x)$, which turns out to detect parity errors for all message sizes N, while the range of two-bit detections is reduced only by a factor of 2.

A number of "standard" CRC polynomials were chosen by no other criteria, sometimes adding only the criterion that they should have only a small number of terms. (This was at one time important for hardware design.) For example, the CCITT (Comité Consultatif International Télégraphique et Téléphonique) has anointed $x^{16} + x^{12} + x^5 + 1$ as "CCITT-16"; it is the product of $x + 1$ and a primitive polynomial. The polynomial ANSI-16 (see table on p. 1169) also has this character.

Similarly for some choices other than 16 bits: "CRC-12" is $(x + 1)(x^{11} + x^2 + 1)$, the latter factor being primitive. The most common 32-bit CRC, "CRC-32," used in the ethernet standard (IEEE 802.3) and elsewhere, is $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$, which is primitive.

Now here is something relatively new in this ancient field [3]: For carefully chosen generators G, all two-bit errors in a packet with payload size N can be detected *even if* $e < N$. This is because the previous argument was sufficient, but not necessary: A cleverly chosen G can fail to divide $x^k - 1$ for other reasons than having a primitive factor of large order. This idea opens up the design space to search, essentially by brute force, for generators that have $D > 2$, that is, are capable of finding not just all two-bit errors, but all three-bit errors, all four-bit errors, etc., up to some bound that depends on N and M. Several of these "new" generators are shown in the table on p. 1169, which is based on [3] (which see for details), along with their recommended values of N. A generator that is good for large N is not necessarily good for small N, and vice versa, so you should stick to the recommended values. The hexadecimal values in the table give binary representations of the polynomials, with the convention that each must be prefaced by a leading 1 (the x^{16} term).

In most protocols, a transmitted block of data consists of some N data bits, directly followed by the M bits of their CRC (or the CRC XORed with a constant; see below). There are two equivalent ways of validating a block at the receiving end. Most obviously, the receiver can compute the CRC of the data bits, and compare it to the transmitted CRC bits. Less obviously, but more elegantly, the receiver can simply compute the CRC of the total block, with $N + M$ bits, and verify that a result of zero is obtained. Proof: The total block is the polynomial $Sx^M + C$ (data left-shifted to make room for the CRC bits). The definition of C is that $Sx^m = QG + C$, where Q is the discarded quotient. But then $Sx^M + C = QG + C + C = QG$ (remember modulo 2), which is a perfect multiple of G. It remains a multiple of G when it gets multiplied by an additional x^M on the receiving end, so it has a zero CRC, q.e.d.

A couple of small variations on the basic procedure need to be mentioned [1]: First, when the CRC is computed, the M-bit register need not be initialized to zero.

Initializing it to some other M-bit value (e.g., all 1's) in effect prefaces all blocks by a phantom message that would have given the initialization value as its remainder. It is advantageous to do this, since the CRC described thus far otherwise cannot detect the addition or removal of any number of initial zero bits. (Loss of an initial bit, or insertion of zero bits, are common "clocking errors.") Second, one can add (XOR) any M-bit constant K to the CRC before it is transmitted. This constant can either be XORed away at the receiving end, or else it just changes the expected CRC of the whole block by a known amount, namely the remainder of dividing G into Kx^M. The constant K is frequently "all bits," changing the CRC into its ones complement. This has the advantage of detecting another kind of error that the CRC would otherwise not find: deletion of an initial 1 bit in the message with spurious insertion of a 1 bit at the end of the block.

The following object `Icrc` implements the calculation of 16-bit CRCs for the generators listed in the table. The constructor sets which generator is to be used, and also whether the initial register should be all bits (the default) or zero. `Icrc` is loosely based on the function in [2]. Here is how to understand its operation: First look at the function `icrc1`. This is used only by the constructor, to initialize a table of length 256, incorporating one character into a 16-bit CRC register. The only trick used is that a character's bits are XORed into the most significant bits of the register, all eight together, instead of being fed into the least significant bit, one bit at a time, at the time of the register shift. This works because XOR is associative and commutative — we can feed in character bits *any* time before they will determine whether to zap with the generator polynomial.

Now look at the methods `crc` and `concat`. Go back to thinking about a character's bits being shifted into the CRC register from the least significant end. The key observation is that while 8 bits are being shifted into the register's low end, all the generator zapping is being determined by the bits already in the high end. Since XOR is commutative and associative, all we need is a table of the results of all this zapping, for each of 256 possible high-bit configurations. Then we can play catch-up and XOR an input character into the result of a lookup into this table. But this is exactly the table that was constructed by `icrc1`. References [2,4,5] give further details on table-driven CRC computations.

icrc.h

```
struct Icrc {
```

Object for computing 16-bit cyclic redundancy checksums.

```
    Uint jcrc,jfill,poly;
    static Uint icrctb[256];

    Icrc(const Int jpoly, const Bool fill=true) : jfill(fill ? 255 : 0) {
```

Constructor. Choose one of 8 generators (see table) by the value of `jpoly`. Initialize the CRC register to all bits if `fill` is true, otherwise to zero.

```
        Int j;
        Uint okpolys[8] = {0x755B,0xA7D3,0x8005,0x1021,0x5935,0x90D9,0x5B93,0x2D17};
```

Generator polynomials, see table.

```
        poly = okpolys[jpoly & 7];
        for (j=0;j<256;j++) {
            icrctb[j]=icrc1(j << 8,0);
```

Table of CRCs of all characters.

```
        }
        jcrc = (jfill | (jfill << 8));
    }
```

```
    Uint crc(const string &bufptr) {
    Initialize the CRC register, compute and return the 16-bit CRC for the string bufptr.
        jcrc = (jfill | (jfill << 8));
        return concat(bufptr);
    }

    Uint concat(const string &bufptr) {
    Without reinitializing the CRC register, compute and return the 16-bit CRC for the string
    bufptr. In effect, this appends bufptr to previous strings since the last call of crc and
    returns the overall CRC.
        Uint j,cword=jcrc,len=bufptr.size();
        for (j=0;j<len;j++) {       Loop over the characters in the string.
            cword=icrctb[Uchar(bufptr[j]) ^ hibyte(cword)] ^ (lobyte(cword) << 8);
        }
        return jcrc = cword;
    }

    Uint icrc1(const Uint jcrc, const Uchar onech) {
    Given a remainder up to now, return the new CRC after one character is added. Used by
    Icrc to initialize its table.
        Int i;
        Uint ans=(jcrc ^ onech << 8);
        for (i=0;i<8;i++) {             Here is where 8 one-bit shifts, and some XORs
            if (ans & 0x8000) ans = (ans <<= 1) ^ poly;      with the generator poly-
            else ans <<= 1;                                  nomial, are done.
            ans &= 0xffff;
        }
        return ans;
    }

    inline Uchar lobyte(const unsigned short x) {
        return (Uchar)(x & 0xff); }
    inline Uchar hibyte(const unsigned short x) {
        return (Uchar)((x >> 8) & 0xff); }
};
Uint Icrc::icrctb[256];
```

What if you need more than 16 bits of checksum? For a true 32-bit CRC, you will need to rewrite the routines given to work with a longer generating polynomial. For example, $x^{32} + x^7 + x^5 + x^3 + x^2 + x + 1$ is primitive modulo 2 and has nonleading, nonzero bits only in its least significant byte (which makes for some simplification). The idea of table lookup on only the most significant byte of the CRC register goes through unchanged.

Easier, if you don't care about the M-consecutive bit property of the checksum, is to just instantiate more than one copy of `Icrc`, each with a different generator (first argument in constructor). These provide statistically independent checks.

22.4.1 Other Kinds of Checksums

Quite different from CRCs are the various techniques used to append a decimal "check digit" to numbers that are handled by human beings (e.g., typed into a computer). Check digits need to be proof against the kinds of highly structured errors that humans tend to make, such as transposing consecutive digits. Wagner and Putter [6] give an interesting introduction to this subject, including specific algorithms.

Checksums now in widespread use vary from fair to poor. The 10-digit ISBN (International Standard Book Number) that you find on most books, including this

one, uses the check equation

$$10d_1 + 9d_2 + 8d_3 + \cdots + 2d_9 + d_{10} = 0 \pmod{11} \tag{22.4.2}$$

where d_{10} is the right-hand check digit. The character "X" is used to represent a check digit value of 10. Another popular scheme is the so-called "IBM check," often used for account numbers (including, e.g., MasterCard). Here, the check equation is

$$2\#d_1 + d_2 + 2\#d_3 + d_4 + \cdots = 0 \pmod{10} \tag{22.4.3}$$

where $2\#d$ means, "multiply d by two and add the resulting decimal digits." United States banks code checks with a nine-digit processing number whose check equation is

$$3a_1 + 7a_2 + a_3 + 3a_4 + 7a_5 + a_6 + 3a_7 + 7a_8 + a_9 = 0 \pmod{10} \tag{22.4.4}$$

The familiar 12-digit Universal Product Code (UPC) is printed with both a decimal representation and a synonymous bar code. The digits are divided into a one-digit "category," a five-digit manufacturer, a five-digit product identification, and one-digit checksum. The check equation is

$$3a_1 + a_2 + 3a_3 + a_4 + 3a_5 + \cdots + 3a_{11} + a_{12} = 0 \pmod{10} \tag{22.4.5}$$

The bar code put on many envelopes by the U.S. Postal Service is decoded by removing the single tall marker bars at each end and breaking the remaining bars into six or ten groups of five. In each group the five bars signify (from left to right) the values $7, 4, 2, 1, 0$. Exactly two of them will be tall. Their sum is the represented digit, except that zero is represented as $7 + 4$. The five- or nine-digit zip code is followed by a check digit, with the check equation

$$\sum d_i = 0 \pmod{10} \tag{22.4.6}$$

None of these schemes is close to optimal. An elegant scheme due to Verhoeff is described in [6]. The underlying idea is to use the ten-element *dihedral group* D_5, which corresponds to the symmetries of a pentagon, instead of the cyclic group of the integers modulo 10. The check equation is

$$a_1 * f(a_2) * f^2(a_3) * \cdots * f^{n-1}(a_n) = 0 \tag{22.4.7}$$

where $*$ is (noncommutative) multiplication in D_5, and f^i denotes the ith iteration of a certain fixed permutation. Verhoeff's method finds *all* single errors in a string, and *all* adjacent transpositions. It also finds about 95% of twin errors ($aa \to bb$), jump transpositions ($acb \to bca$), and jump twin errors ($aca \to bcb$). Here is an implementation:

decchk.h

```
Bool decchk(string str, char &ch) {
```

Decimal check digit computation or verification. Returns as `ch` a check digit for appending to `string[0..n-1]`, that is, for storing into `string[n]`. In this mode, ignore the returned boolean value. If `string[0..n-1]` already ends with a check digit (`string[n-1]`), returns the function value `true` if the check digit is valid, otherwise `false`. In this mode, ignore the returned value of `ch`. Note that `string` and `ch` contain ASCII characters corresponding to the digits 0-9, *not*